Self-Adaptive PLCA for 10BASE-T1S Zonal Automotive Networks

Nov 4, 2026 · @Thejaselhou Lhoungu

Abstract. 10BASE-T1S shares one cable among many nodes using PLCA, a fair round robin that ignores urgency. We ask whether a coordinator can retune only standard PLCA parameters at run time, guided by a small learned model, without losing a deterministic latency bound. We built the design in OMNeT++ 6.4 with INET 4.7 as a 9-node zonal segment, trained a 596-byte int8 network on 467,844 simulated windows, and ran 1,247 simulations with 10 seeds per point in the comparisons. A governor that checks every proposed change against a worst-case bound gave 0 safety deadline misses in all 480 governed comparison runs, including runs fed random predictions. Without the governor, the same policy let safety frames reach 35.3 ms against a 20 ms deadline. Conventional PLCA missed the 20 ms deadline for 20 % of safety frames at 90 % load. The learned model did not beat a plain moving average: at 90 % load their difference in p99 latency stayed within the 95 % confidence interval. Against fixed safety priority, adaptation cut p99 latency by 16.6 % at 90 % load, raised mean latency by 4.6 %, and gave no p99 gain on bursty traffic. PLCA wastes little idle time, so burst changes move delay between nodes rather than add capacity. The useful result is the governor, which makes run-time retuning safe. All results are simulated.

1. Introduction

PLCA gives every node a fair turn on a shared 10BASE-T1S cable, but it cannot tell a brake message from a radio message. Zonal vehicles put both on the same segment, so this gap now matters in production designs. Contributors to IEEE 802.3 state that PLCA latency is bounded but "too long" for control loops, and a study on reducing it was proposed in 2025 [3], [4].

Modern vehicles are moving from one ECU per function to zonal architectures, where a central computer connects to zone controllers. Inside a zone, many small devices share a cheap link. 10BASE-T1S provides that link: one twisted pair, many nodes, 10 Mb/s, standardised in IEEE 802.3cg [1].

Research question. How can a 10BASE-T1S network understand current and future conditions and adapt its communication policy, while keeping deterministic, bounded guarantees?

Contributions. This paper makes five contributions:

1. An OMNeT++ model of a zonal 10BASE-T1S segment whose coordinator retunes standard burst settings, with baselines for five traffic scenarios.

2. A deterministic governor for standards-only retuning, with a worst-case bound that holds whatever the predictor outputs.
3. A simulated dataset of 467,844 labelled network-state windows, because we know of no public 10BASE-T1S dataset, and a comparison of tiny predictors on it.
4. An evaluation against four baselines, with ablation and sensitivity studies and 95 % confidence intervals.
5. A replay of real CAN traces from CarDS and ROAD onto a simulated zone.
6. A measured finding that the governor carries the value, while prediction adds no detectable gain over measurement.

2. Background

PLCA turns a shared half-duplex bus into a collision-free round robin, so its worst case depends on the frames other nodes send. PLCA is an optional reconciliation sublayer defined in clause 148 of IEEE 802.3cg [1], [2]. It is not a replacement for TSN, which is expected to run on top of it [2].

The PLCA cycle. Node 0, the coordinator, sends a BEACON to start a cycle [2]. Each node then gets one transmit opportunity (TO) in node-ID order. A node with data transmits; a silent node's TO expires after a short timer, so idle nodes cost almost nothing.

Burst mode. A node may be allowed to send several frames in one TO [1], [2]. This setting is normally static and the same for every node. Longer bursts help the sender but lengthen the cycle for everyone else [6].

The worst case. With N nodes all sending maximum-size frames, one cycle lasts about N × 1.23 ms. The IEEE study group quotes 9.83 ms for 8 nodes [3]. A frame behind q others in its node's queue waits about q cycles [6], so a queue directly multiplies this cycle time.

Mixed criticality. PLCA serves nodes, not messages. A safety message therefore has no more access to the bus than a software-update frame on another node. Its waiting time rises with traffic it has nothing to do with.

3. Related work

Deterministic PLCA is an active field, but existing proposals change the protocol, rely on a vendor feature or use a fixed schedule. We found none that keeps standard PLCA and retunes it at run time behind a checked bound. Our searches were broad but cannot be exhaustive, so this claim should be re-checked before submission. The IEEE P802.3da task force is also working on multidrop enhancements [27]; we did not read its drafts.

Method	Adaptive?	Predictive?	Determinism guarantee?	Standard-compliant?
Burst mode, static tuning [1], [6]	No	No	Design-time bound	Yes
Kim et al.: simulated 802.3cg performance [10]	No	No	No	Yes
Cerruto et al.: zonal simulation [9]	No	No	Simulation only	Yes
Cerruto et al.: application-level shaping [21]	No	No	Not confirmed	Yes
Cerruto et al.: FS2 segmentation and shaping [22]	Not confirmed	No	Not confirmed	Yes, as stated by its authors
do Nascimento et al.: Time-Aware Shaper on PLCA [19]	No	No	Yes, delay bounds	Standard TSN above PLCA
Pannell: Priority Advertise [5]	Per beacon	No	No	No, proposed change
Min and Park: prioritisation and segmentation [7]	No	No	Lower latency, analysed	No
Min and Park: PLCA with interrupts [8]	No	No	More predictable latency	No
Jochim et al.: cycle-gated differentiated services [3], [4], [23]	No	No	Bounded latency is the aim	No, proposed change
Ahn and Park: Periodic PLCA [20]	No	No	Yes, jitter bound	No, changes the RS
IC Plus Corp.: emergency interference packet, EP 4 401 363 [25]	Event-driven	No	None given	No, nodes inject interference

Method	Adaptive?	Predictive?	Determinism guarantee?	Standard-compliant?
onsemi: Precedence Mode [26]	No	No	Not stated	Vendor extension
Ciraolo: PLCA and CSMA/CD on a test network [28]	No	No	No	Yes
Proposed in this work	Yes	Yes	Yes	Yes, parameters only; run-time changes to be checked against the standard text

Latency bounds. Our per-flow arrival limit is a simple arrival curve in the sense of network calculus [11], [12]. Davis et al. [13] showed that an accepted CAN analysis was flawed despite years of use, which is why we test our bound against simulation as well as prove it.

TinyML. Surveys cover machine learning on microcontroller-class hardware [14], [15]. We compare a shallow decision tree, a random forest and a small int8 neural network, all exported to plain C with no runtime library.

Datasets. CarDS provides real CAN and Automotive Ethernet traces [24], and ROAD provides real CAN traffic from one vehicle [16]. AutoHack [17] and can-train-and-test [18] could extend this study. TOW-IDS, listed in the project plan, needs a paid IEEE DataPort subscription, so we did not use it. We know of no public 10BASE-T1S traffic dataset.

4. Problem analysis

PLCA's problem in mixed traffic is the order of service, not capacity. Three properties set what any design inside the standard can achieve.

Little capacity is wasted. A silent node costs one 32 BT timeout, about 3 µs, so the cycle shrinks when nodes have nothing to send. PLCA is therefore close to work-conserving. A longer burst for one node adds no capacity; it moves delay from that node to the others. Any gain from retuning has to come from this reallocation.

The worst case grows with every burst allowance. With b(i) extra frames allowed at node i, the longest cycle is bounded by the first line below. A safety frame waits at most one cycle for its TO and is then sent inside its node's burst, which gives the second line.

```
T_{cycle,max} \le T_{beacon} + \sum_i \left[(1 + b_i) T_{frame,max,i} +
T_{ifg}\right]
```

```
L_{safety,max} \le T_{cycle,max} + T_{frame,max} \le D_{safety}
```

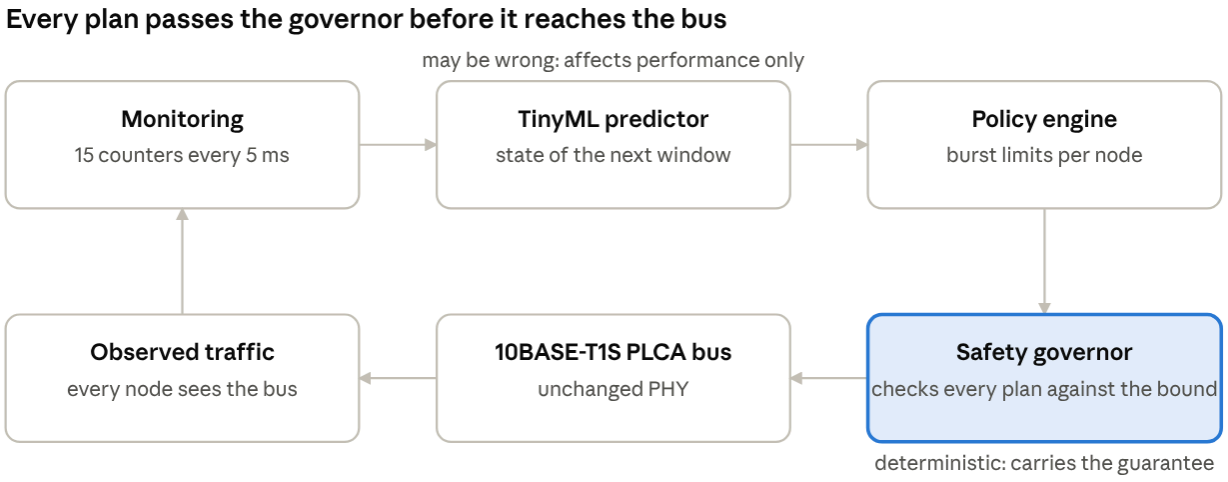
A frame of 1518 bytes takes 1.23 ms on the bus. In a zone with six such nodes one cycle already lasts about 7.4 ms before any safety traffic is counted, so a deadline of 2 ms cannot be guaranteed by standard parameters alone. Section 7.14 shows what a stricter, non-standard meter would take.

A node's turn cannot follow its queue. Safety frames share a cycle with everything else, so frames arrive faster than one per cycle when cycles are long. A node sending every 1 ms needs a burst of at least one frame per millisecond of cycle, or its queue grows without limit.

The knobs. Burst count is the only setting we change at run time. INET reads it at each burst decision, while it reads node IDs only at start-up, so node order is fixed per run here. Node count, TO timer and burst timer stay at their static values.

5. Proposed framework

A learned model proposes and a small deterministic governor decides, so a wrong prediction can cost performance but never a deadline. Every plan passes through the governor before it reaches the bus.



framework · feedback loop with one deterministic gate

The top row may be wrong without harm, because only the governor's limits enter the proof below.

Monitoring. Every 5 ms the coordinator records 15 cheap counters: bus and TO utilisation, arrival rate, queue lengths, critical frames waiting, cycle length, their trends and smoothed values.

TinyML predictor. A network with two hidden layers of 16 units predicts the state of the next window: low (under 30 % bus use), medium (under 60 %), high (under 85 %), or congested (85 % or more, or 10 or more frames waiting). It is quantised to int8, takes 596 bytes of parameters and is exported to plain C.

Policy engine. Low and medium states use the base settings. In the high state, background nodes with 2 or more queued frames get 1 extra frame per TO, and in the congested state 3 extra. A new state must hold for two windows before the policy acts, which damps oscillation.

5.1 Standards-only governor

The governor checks each proposal and adds nothing to the protocol. It refuses any proposal that could break the bound and keeps the last safe setting. Let P be the safety period, S the bus time of one safety frame and D the deadline.

Fixed safety room. The room R is the largest number of safety frames that fits, R = floor(D / (P + S)). Safety nodes get a burst of R - 1 and that value never changes: lowering it could strand a backlog built up in a long cycle. For D of 10, 15, 20 and 30 ms, R is 9, 14, 18 and 28.

Acceptance test. A proposal with worst cycle C is accepted only if both conditions hold:

```
\lceil C / P \rceil \le R \qquad C + R \cdot S \le D
```

The first condition says at most R safety frames arrive during one cycle, so the safety queue empties at every TO. The second is the plan's latency bound, with the frames ahead in the burst counted. Together they bound every safety frame by C + R S.

Cycle boundary. Accepted settings take effect only when a new cycle starts. An earlier version changed them mid-cycle, which could mix two settings into a cycle longer than either. That version also lowered safety bursts; together these broke the bound in 14 of 160 test runs, and both rules above came from fixing it.

At D = 20 ms the base cycle is 9.8 ms, the accepted limit is 18 ms and the bound is 19.2 ms. At D = 10 ms the base cycle is 8.6 ms and no extra 1518-byte frame fits, so the governor has nothing to give and the design equals fixed priority.

5.2 Metered governor

For deadlines near 2 ms we also studied a stricter variant in the cycle-level simulator. It meters non-critical bus time inside each cycle, so nodes must send less than PLCA would allow. This goes beyond a parameter change, so it is a comparison point, not the main design.

Safety governor. The governor fixes two limits at design time. Each critical node i may send L(i) critical frames per TO, and non-critical traffic may use at most B bit times between two TOs of any critical node. Every node sees the bus, so each can enforce B locally.

Let F be the fixed overhead of one cycle, c(i) the bus time of one critical frame, D the tightest deadline scaled by a margin of 0.95, and L(i) the largest number of frames of flow i in any window of length D:

```
F = T_{beacon} + N \cdot T_{TO} \qquad C = F + \sum_j L_j c_j + B
```

```
B = D - F - \sum_j L_j c_j - \max_j (L_j c_j)
```

Theorem. If each critical flow respects L(i), every critical frame is delivered within C + L(i) c(i), which is at most D.

Proof. Between the starts of two consecutive TOs of a critical node there is one BEACON, one TO of each node, at most L(j) critical frames of each critical node j, and at most B of non-critical time. The gap is therefore at most C. Because C is at most D, at most L(i) frames of flow i arrive within one gap, so the queue is cleared at every visit. A frame arriving just after a TO starts waits at most C, then takes at most L(i) c(i) to send. ∎

The proof uses only the limits L and B, never the prediction. If the flows are strictly periodic with period P, then L = ceil(D/P), which gives 1.9 ms for the mixed scenario. If B cannot fit one maximum-size frame, the governor refuses the configuration at design time. With no critical flows it caps the gap at conventional PLCA's own worst case, so adaptation cannot lengthen it.

6. Methodology

The main evaluation runs in OMNeT++ 6.4 with INET 4.7. We extended its PLCA model with a coordinator module, AdaptivePlcaController, and changed nothing in PLCA's behaviour beyond the burst count. Real traces were replayed in a separate cycle-level Python simulator.

Network. One coordinator and 8 followers on a multidrop segment, with 4, 12 and 16 followers in the sweep. BEACON is 20 BT, an empty TO 32 BT and a bit time 100 ns. A frame takes (bytes + 8) × 8 + 96 BT on the bus. Each node has one 64-frame drop-tail queue. All followers send to the coordinator, which stands for the zone's uplink.

Scenarios. Offered load is a share of 10 Mb/s.

Scenario	Traffic
S1 low, S2 rising load	Poisson, equal on all followers, frames of 64 to 1518 bytes, load 25 to 90 %
S3 bursty	30 % of load Poisson, 70 % in bursts of about 8 frames
S4 mixed	2 safety nodes send 64-byte frames every 1 ms; 6 background nodes send 512 to 1518 bytes
S5 mixed and bursty	S4 safety flows; background in bursts of about 8 frames

Designs compared.

Label	Design
B1	Conventional PLCA, no burst
B2	Best static setting, chosen offline per load from two candidates near the safety room
B3	Fixed priority: safety nodes first with burst R - 1, background without burst
B4	Reactive: state from a moving average of utilisation, with the same governor
P	Proposed: predicted state and the governor

The ablation removes the governor (A1), puts safety nodes last (A2) and feeds random predictions (A3). The sensitivity study varies window size, model type and deadline.

Metrics. Mean and p99 latency, safety deadline miss ratio, worst safety latency, frame loss, the share of proposals refused and accepted reconfigurations per second. A safety frame that is lost or still queued past its deadline counts as a miss. Each point has 10 seeds, 60 s of simulated time and a 5 s warm-up. Intervals are 95 % Student t. P and its baselines share seeds, so we also report same-seed differences.

Deadline. We use 20 ms for comparisons, because at 10 ms the governor leaves no room to adapt. No public source gives real deadlines for these flows, so this is an assumption, and we tested 10, 15 and 30 ms as well.

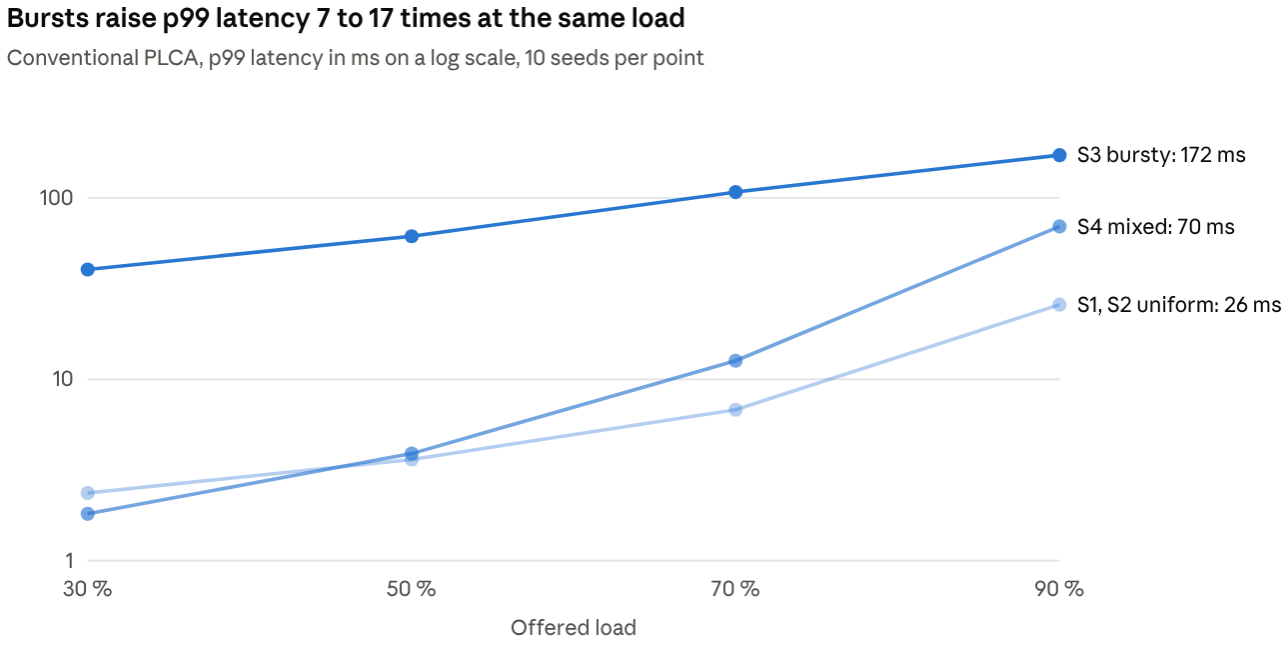
Dataset and training. Three scenarios at every load gave 467,844 windows of 5 ms with 15 features, recorded under conventional PLCA. Two seeds per load trained the models and a third, never seen, tested them. The class shares are 21.7, 28.0, 16.6 and 33.7 %.

Real traffic. We replayed 9 of 39 profiled CarDS traces and 12 ROAD logs through the cycle-level simulator, not through OMNeT++. Neither dataset says which messages are safety-critical, so the critical flows are an assumption.

7. Results

The governor held every bound. Conventional PLCA failed under load, while adaptation gave a modest p99 gain in one scenario and prediction gave no detectable gain over measurement.

7.1 Baseline



results/omnet/summary.csv · configs S1_S2_Uniform, S3_Bursty, S4_Mixed

Conventional PLCA holds up on smooth traffic and degrades sharply on bursts and on mixed traffic. Bursts raise p99 latency 7 to 17 times at the same load. The OMNeT++ means at 25 to 90 % load agree with the cycle-level simulator within about 2.5 %, which is a useful cross-check on both models.

In S4 the worst safety latency reached 221.5 ms at 90 % load, and 2.4 % of safety frames passed 10 ms already at 70 % load.

Followers at 90 % load	Mean (ms)	p99 (ms)	Longest cycle (ms)
4	3.96	23.0	4.89
8	4.05	25.8	7.26
12	3.95	24.5	9.41
16	4.03	25.3	10.76

Average latency barely depends on the number of nodes at a fixed total load, but the longest cycle grows with it, and that is what limits a guarantee.

7.2 Predicting the network state

The int8 network was the smallest model that clearly beat the two no-ML methods, and it nearly matched an oracle that knows the true scenario and load. About a third of the windows, from the unseen seed, form the test set.

Predictor	Accuracy	Macro F1	Parameters
Oracle: true scenario and load	65.4 %	0.624	none
MLP 2×16, float32	64.6 %	0.623	2,384 bytes
MLP 2×16, int8	64.2 %	0.619	596 bytes
Gradient boosting (reference)	63.2 %	0.619	not sized
Random forest, 20 trees	60.9 %	0.596	20,288 bytes
Moving average threshold, no ML	58.0 %	0.565	none
Persistence: next equals now	58.0 %	0.555	none
Decision tree, depth 4	57.0 %	0.542	248 bytes

The gap between the oracle and persistence is only 7 points, because the near future of a 5 ms window is largely random. Compiled for x86-64, the MLP takes 3,652 bytes and 321 ns per inference, and the tree 224 bytes and 10 ns. We did not build them for a microcontroller, so these figures are indicative only. The exported C matches the Python models.

7.3 The five designs

Every design with a safety burst met the 20 ms deadline at 90 % load, and P had the lowest p99 of them, though its interval overlaps B4's. The table is S4 at 90 % load, 10 seeds.

Design	Safety misses at 20 ms	p99 (ms)	Mean (ms)	Refused	Changes per s
B1 conventional	20.0 ± 1.7 %	69.6 ± 5.0	8.93	n/a	0
B2 best static	0 %	17.8 ± 0.6	3.11	n/a	0
B3 fixed priority	0 %	19.1 ± 1.0	2.53	n/a	0
B4 reactive	0 %	16.8 ± 0.8	2.72	3.9 %	49
P proposed	0 %	15.9 ± 1.0	2.65	3.4 %	49

Same-seed differences give the fairer test, shown as P minus the baseline.

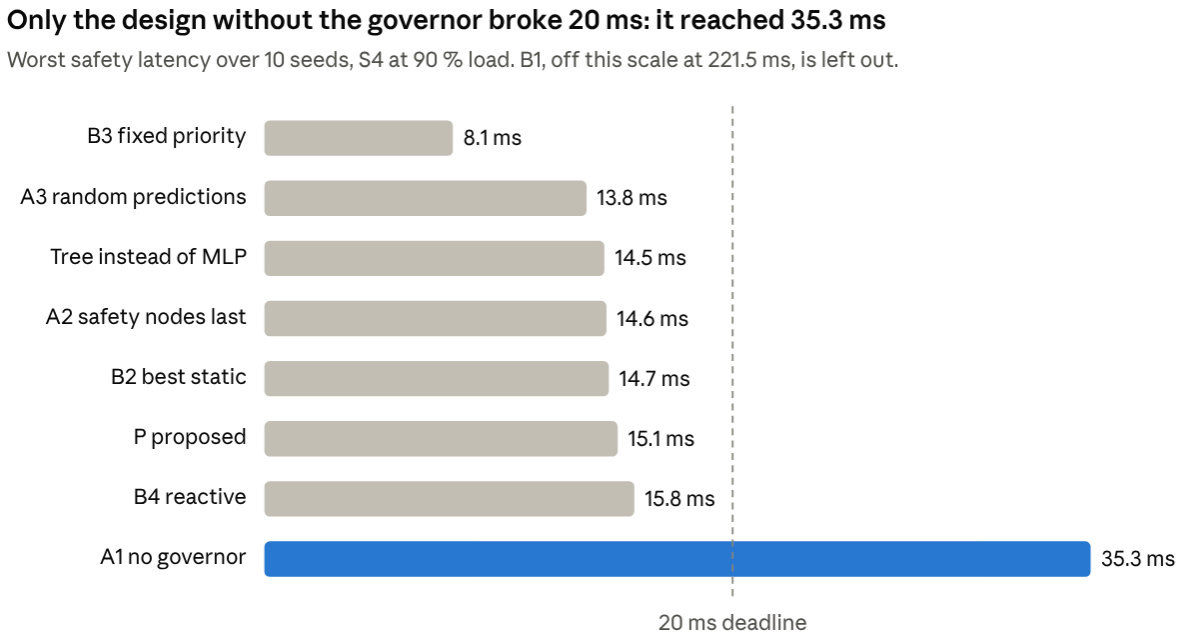
Comparison	Load	Mean	p99
P against B3, S4	70 %	+1.9 % (borderline)	+2.9 % (borderline)
P against B3, S4	90 %	+4.6 % (significant)	-16.6 % (significant)
P against B4, S4	90 %	-2.7 % (not significant)	-5.4 % (not significant)
P against B3, S5	90 %	+6.7 % (significant)	+4.6 % (not significant)
P against B4, S5	90 %	+3.3 % (not significant)	+5.7 % (not significant)

At 30 and 50 % load P and B3 do not differ, because the policy rarely acts. At 90 % in S4, P trades a small rise in the mean for a shorter tail. No comparison between P and B4 was significant at any load, in S4 or S5, so prediction adds no detectable gain over measurement here.

At 10 ms the two designs are identical. Every result of P at 10 ms matched B3 to all reported digits, with 0 changes and up to 27 % of proposals refused. The governor correctly left nothing to adapt.

A static tuning can break the deadline. B2's better candidate met 20 ms, but the other, with safety bursts of 34, reached 26.4 ms at 90 % load. Offline tuning needs the same check.

7.4 Ablation



results/omnet/summary.csv · 90 % load, worst of 10 seeds

The governor is the part that matters. Without it, the same policy missed 0.073 ± 0.054 % at 20 ms and cycles reached 24.4 ms. With it, random predictions (A3) caused no miss.

A3's p99 of 17.7 ms was still below B3's 19.1 ms, because at 90 % load almost any extra background burst shortens the tail a little. Putting safety nodes last (A2) cost little, with p99 of 16.6 ms against 15.9 ms and intervals that overlap.

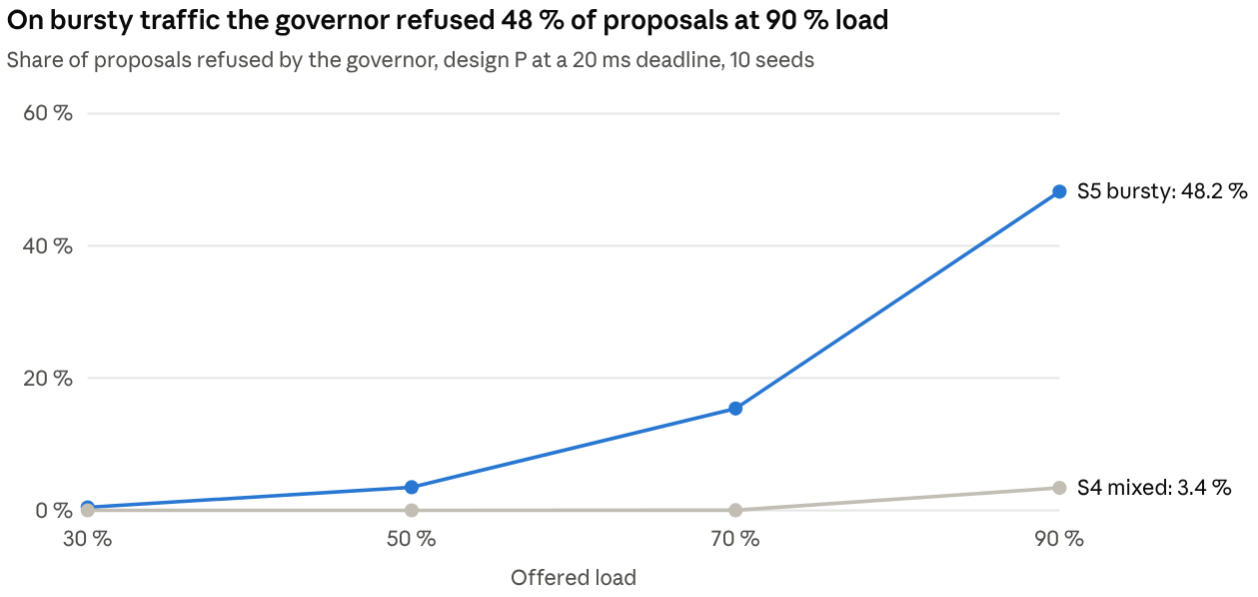
7.5 Sensitivity

No setting changed the conclusions. All rows are S4 at 90 % load with no safety miss.

Setting	p99 (ms)	Mean (ms)	Refused	Changes per s	Worst safety (ms)
Window 2 ms	16.6 ± 1.0	2.77	4.0 %	61	14.8
Window 5 ms (P)	15.9 ± 1.0	2.65	3.4 %	49	15.1
Window 20 ms	17.3 ± 0.7	2.59	3.8 %	19	14.8
Tree instead of MLP	17.0 ± 1.3	2.73	4.1 %	51	14.5
Deadline 15 ms	17.8 ± 0.8	2.63	10.4 %	41	11.1
Deadline 30 ms	17.2 ± 0.9	2.85	0.5 %	52	20.9

A looser deadline gives the policy more freedom, yet p99 did not improve at 30 ms and the mean got worse. More freedom to lengthen bursts is not more performance, which fits Section 4. The window sizes of 2 and 20 ms reuse the model trained on 5 ms windows, so those two rows are a weaker test.

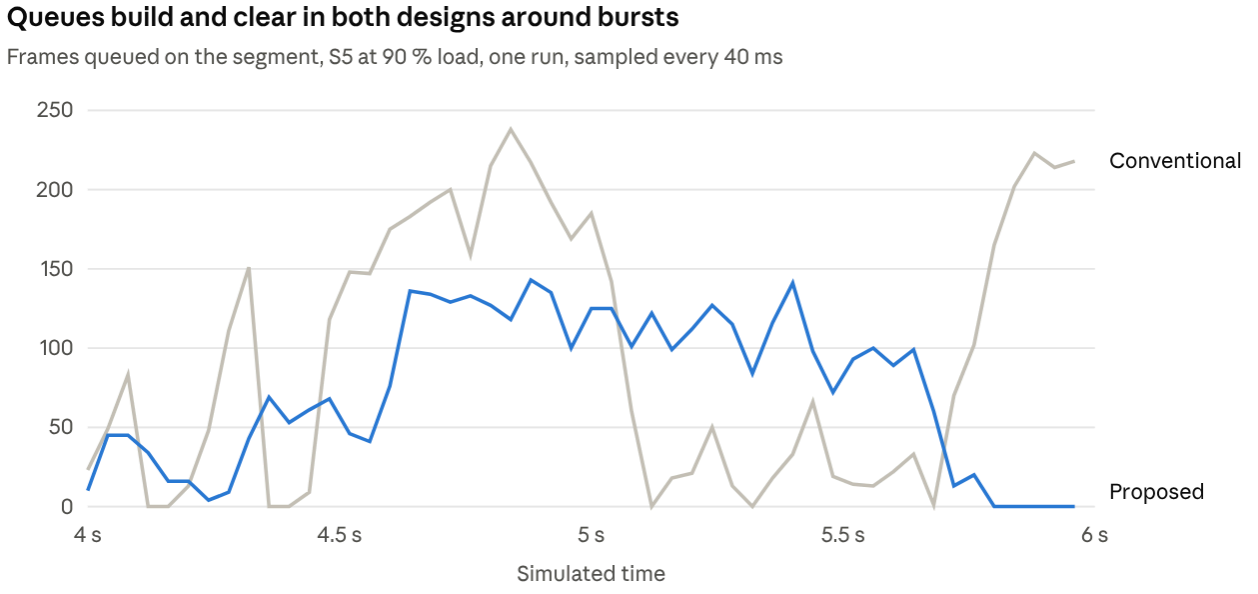
7.6 Bursty traffic and overhead



results/omnet/summary.csv · configs P20 and S5_P20

On S5 at 90 % load B1 missed 55.1 % at 20 ms, with a worst case of 385.8 ms and 15 % frame loss. B3, B4 and P all had 0 misses, with worst safety latencies of 8.1, 15.6 and 15.9 ms. Adaptation did not help here: P's mean rose against B3 at every load, by 6.7 to 14.4 %, and its p99 did not differ significantly. The governor refused 48 % of proposals at 90 % load, so most of what the policy asked for could not be guaranteed.

Accepted changes peaked near 54 per second. The simulation charges no time for sending a new setting to a follower, so this overhead is a lower bound.



results/omnet/timeline/T_Conventional-3.csv and T_Proposed-3.csv · one run each

This is one run of 10 s, so it illustrates behaviour rather than proves a difference: queues build and clear in both designs.

7.7 Real CAN traffic: CarDS

CarDS traffic alone is light on a 10BASE-T1S zone. Across 39 traces and 21.4 bus hours, per-bus load was 0.1 to 13 %, and the busiest 10 ms stayed at or below 31 %. We replayed 9 traces on 4 assumed zones with 3 seeds, using 64-byte frames and the metered governor.

With CAN alone, at 9.8 to 21 % load, no design missed any deadline. With 30 % added bulk traffic, itself an assumption, conventional PLCA missed a 2 ms deadline in 104 of 108 runs, with a worst latency of 2.91 ms. The metered governor had 0 misses in 108 runs and a worst latency of 1.78 ms. At 5 and 10 ms neither design missed. These results agree with ROAD below: on real traffic the benefit depends on the deadline and on bulk traffic sharing the bus.

7.8 Real CAN traffic: ROAD

On 12 real ROAD logs the governor had no critical miss in 1,620 runs, stayed within its 1.9 ms bound, and refused the one load it could not guarantee. Conventional PLCA missed a 2 ms deadline in 271 of its 600 runs, all with bulk Ethernet present. CAN traffic alone was easy for PLCA, and at deadlines of 5 ms or more conventional PLCA missed nothing (Section 7.13).

7.9 ROAD: method

Each ambient log's first 300 s is read with real timestamps, and random windows of 2 s are replayed for 5 seeds. Every CAN frame becomes one 64-byte Ethernet frame, a deliberately plain migration with no frame aggregation.

- **Zone load.** A log replayed k times with independent offsets stands for a zone collecting k CAN segments.
- **Node mapping.** ROAD's DBC is anonymised, so sending ECUs are unknown. Critical flows are spread over nodes first; other IDs are balanced by load.
- **Critical flows.** We choose the fastest periodic IDs using the lowest ID only as a repeatable tie-break, and give them a 2 ms deadline. In all 12 logs these are 0x00E and 0x033 at about 100 Hz. ROAD does not say which messages are safety-critical, and its ID obfuscation does not preserve arbitration priority [16], so this choice is an assumption, tested in Section 7.13.
- **Experiments.** A is CAN only, with 1 to 8 segments. B adds 30 % bulk Ethernet of 512 to 1518-byte frames on 2 nodes, with 1 to 5 segments.

7.10 ROAD: two flaws that real data exposed

Our first replay of the real logs broke the guarantee, which synthetic traffic had never done. Both causes are general lessons for trace-driven work.

1. **Percentile periods are unsafe.** The governor first assumed a period equal to the 1st-percentile gap between frames. The first choice of critical ID, 0xFFF, is the most frequent ID, but ROAD uses it as a filler for constant, aperiodic and very slow IDs [16], so its gaps fall to about 1 µs. Latency reached 3.47 ms against a claimed 1.9 ms bound. L is now the largest count in any deadline-long window of the trace.
2. **Delivered-only metrics hide losses.** In one overloaded replay of 8 segments the metric reported 0 % misses, while 10 % of critical frames had been dropped from queues full of non-critical frames. Critical frames now have their own queue, and every lost frame counts as a miss.

The fixes are covered by regression tests in the project repository.

7.11 ROAD: results

Experiment A produced no misses under any policy, up to an overloaded 113 % of the bus. Frames of 64 bytes keep the PLCA cycle short, so critical latency stayed at or below 1.14 ms even without the governor.

Experiment B is where the governor is needed. Each row below covers 60 runs: 12 logs by 5 seeds.

CAN segments	Load	Runs with a miss, conventional	Worst critical latency, conventional (ms)	Worst critical latency, governed (ms)	p99 all frames, conventional (ms)	p99 all frames, governor only (ms)
1	42 %	48 of 60	2.91	1.80	34.4	35.0
2	56 %	54 of 60	2.93	1.73	35.1	41.0
3	71 %	57 of 60	2.93	1.72	44.0	65.6
4	84 %	55 of 60	2.93	1.56	49.3	98.6
5	98 %	57 of 60	5.06	Refused	49.9	Refused

Conventional PLCA missed few frames on average, about 0.5 to 0.9 % and at most 2.5 % in a run. For a safety message, that is still a repeated failure, and it occurred at only 42 % load. At 98 % load the 10 critical flows left too little budget for a 1518-byte frame, so the governor rejected the configuration at design time.

The cost is higher on real traffic. Governing nearly doubles p99 latency at 84 % load, and frame loss rises from 0.4 % to 2.6 %. Critical frames are few, so most frames pay this price.

Adaptation did not help here. With bulk traffic at 2 and 3 segments, reactive and TinyML policies raised p99 to 44.1 and 74.0 ms, worse than the governor alone. From 4 segments, every node carries a critical flow, so the governor fixes every limit and the policy has no

freedom left. Replay of real traffic therefore supports the governor, not burst adaptation.

7.12 ROAD: TinyML on real traffic

On real traffic the model needs real training data: a tree trained on ROAD reaches 81.6 %, while the synthetic tree does no better than persistence. Every result below predicts a log that the model never saw, across 40,494 windows from a varied mix of loads.

Model	Accuracy on an unseen log	Macro F1
Tree trained on ROAD only	81.6 %	0.82
Tree trained on synthetic and ROAD	78.3 %	0.77
Persistence	78.2 %	0.77
Tree trained on synthetic only	77.8 %	0.77

Accuracy is higher than on synthetic traffic, probably because real CAN traffic is strongly periodic; the two test sets also differ in their mix of states. Pooling synthetic data with ROAD gave no gain here. It also did not improve accuracy on synthetic test seeds, which fell from 62.4 % to 61.4 %.

Closed loop on held-out logs. Better offline accuracy did not turn into better network behaviour. We held out 3 logs, trained a tree on synthetic data plus the other 9, and replayed the 3 unseen logs. Compared with the synthetic tree, p99 latency stayed within 2 % at every load, and neither tree missed a deadline.

7.13 ROAD: how far the results depend on the assumptions

The guarantee does not depend on which IDs are critical, but the benefit depends strongly on the deadline. Conventional PLCA needs help on this traffic only for deadlines shorter than about 5 ms.

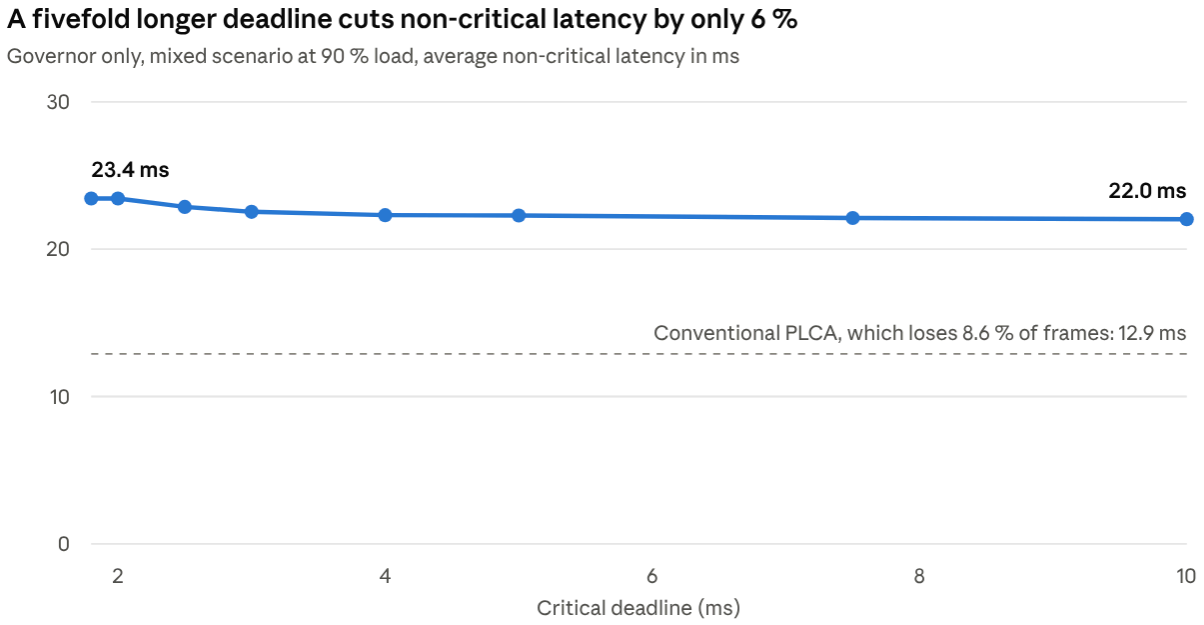
Critical IDs	Deadline	Governed runs with a miss	Worst latency, governed (ms)	Conventional runs with a miss	Worst latency, conventional (ms)
Main choice (Section 7.11)	2 ms	0 of 1,620	1.80	271 of 600	5.06
Next two in order	2 ms	0 of 504	1.74	163 of 288	4.78
Following two	2 ms	0 of 504	1.76	164 of 288	4.29
Main choice	5 ms	0 of 576	4.64	0 of 288	4.37
Main choice	10 ms	0 of 576	8.96	0 of 288	4.37

The last four rows use 3 seeds and fewer load levels. The 5.06 ms in the first row shows that a 5 ms deadline could still be missed rarely.

A longer deadline also removes most of the cost. At 5 ms and 3 CAN segments, the governor alone left p99 unchanged at 42.1 ms. Burst adaptation still raised it, to 63.0 ms. No public source we found gives real deadlines for these messages, because ROAD hides what they are.

7.14 Metered governor: cost and shortest deadline

The budget is not the main cost of the guarantee: relaxing the deadline more than fivefold cuts non-critical latency by only 6 %. Adapting the budget has almost no effect either. These studies use the metered governor in the cycle-level simulator, on the mixed scenario with 5 fresh seeds.



results/tradeoff_deadline.csv · 5 seeds per point

Budget adaptation. We shrank B in low-traffic states, using the predicted state. Non-critical latency changed by under 1 % at 50, 70 and 90 % load, with 0 % misses throughout. There is little room to act: B is 1,469 µs and cannot go below one maximum-size frame of 1,230 µs. The proposal in the original project plan therefore does not reduce the cost, and it was not adopted.

Where the cost comes from. Our reading is that much of the gap comes from conventional PLCA not serving critical traffic. At 90 % load it drops or delays critical frames, which leaves more bus time for everything else, and it loses 8.6 % of frames that then drop out of its latency average. We did not isolate this effect in a dedicated experiment.

Sensitivity. The guarantee held at every setting tested, from 4 to 16 nodes and with TO timers of 20, 32 and 64 BT. Worst critical latency stayed at or below 1.51 ms against the 1.9 ms bound, with 0 % misses. For deadlines from 1.8 to 10 ms, worst latency also stayed under the bound. At a 10 ms deadline the bound is 9.5 ms and the worst seen is 6.7 ms, so it is safe but not tight.

The limit on deadlines. The largest frame on the segment sets the shortest deadline the governor can guarantee, because a frame cannot be interrupted. Min and Park state that safety and powertrain domains need under a few hundred microseconds [7]. Our design reaches that only if every frame on the segment is small.

Largest frame on the segment	One frame on the bus (ms)	Shortest deadline, 2 critical flows (µs)	Shortest deadline, 6 critical flows (µs)
64 bytes	0.07	312	595
128 bytes	0.12	366	649
256 bytes	0.22	474	757
512 bytes	0.43	690	973
1024 bytes	0.84	1,121	1,404
1518 bytes	1.23	1,537	1,820

These values are calculated from the admission test for 8 nodes, with each flow sending one 64-byte frame per deadline.

8. Discussion and limits

The answer to the research question is yes, with a narrower benefit than planned. Standard PLCA parameters can be retuned at run time with a deterministic bound, and the governor is what makes that safe. Prediction did not earn its place: the int8 network met the bound and matched the moving average, and random inputs did not break anything.

The physical reason is in Section 4. PLCA is close to work-conserving, so a burst setting only chooses who waits. Fixed safety priority already removes nearly all of the safety problem, and adaptation can only trade tail against mean among background nodes. A predictor would need something to anticipate that a 5 ms reaction cannot catch, and at this time scale traffic is largely random.

This changes the paper's claim from "prediction improves the network" to "a deterministic governor makes retuning safe". The second claim is the one our data supports.

Deployment. The governor runs on the coordinator and needs no PHY change. Followers must accept a new burst count at run time, and something must carry it to them. We have not built this on hardware, and the simulation gives that step no cost. Whether per-node burst counts changed in operation fit the letter of IEEE 802.3cg still has to be checked against clause 148.

What the guarantee assumes. All of these must hold:

- Every node obeys the budget. One node that ignores it breaks the bound, so policing is needed.
- Critical flows stay within their declared L. On ROAD, L is measured from the trace, which is a replay check rather than a proof for future traffic.
- The bus is fault-free. We model no bit errors, lost BEACONs, node failures or restarts.

Threats to validity.

Threat	Effect	Mitigation or status
Simulation only	Timing details such as PHY delays are simplified	Worst-case cycle matches [3] within 0.2 %; hardware test is future work
ROAD timestamps cluster about 1 ms apart: 63 % of gaps in the highway log are under 100 µs, and ROAD gives 100 µs accuracy [16]	Arrivals may be burstier than on the real bus	Probably pessimistic, but not corrected
ROAD covers one vehicle, mostly on a dynamometer	Results may not generalise	AutoHack and can-train-and-test could extend this
Critical IDs and the 2 ms deadline are assumed	The result depends on them: conventional PLCA misses at 2 ms but not at 5 ms	Other IDs and deadlines tested in Section 7.13; real requirements are still needed
Node mapping is assumed	Real ECU placement is unknown, and 0xFFF stands for several hidden IDs but is placed as one	Balanced by load; a different mapping may change results
Frames are migrated one-to-one	Aggregation would cut the load	Our loads are therefore conservative
TinyML is trained on conventional PLCA windows	Traffic differs once the governor acts	Model errors cannot affect the bound

Threat	Effect	Mitigation or status
Short runs and few seeds in the cycle-level simulator	Mean miss rates vary: 69 % to 78 % at 90 % load across seed sets	Worst cases use every run; rates are indicative

Further limits of the OMNeT++ study.

- Zero misses in 480 runs is evidence, not proof; the guarantee rests on the bound in Section 5.1, under its assumptions.
- Those assumptions include periodic safety flows with known period and frame size, and every node respecting its burst count.
- Node order is fixed per run because of how INET reads node IDs, so we tested reordering only as a static choice.
- S1 to S5 are synthetic. Real traces ran only in the cycle-level simulator, with the metered governor, and we did not use TOW-IDS.
- The models were trained on windows from conventional PLCA, and the sensitivity runs at 2 and 20 ms reuse the 5 ms model.
- Model size and time are x86-64 figures, not microcontroller figures.
- The 20 ms deadline and the 30 % bulk share are assumptions. A deadline of 2 ms is out of reach for the standards-only governor with full-size frames.
- We studied one segment with no bit errors, lost BEACONs or node failures.

9. Conclusion and future work

A deterministic governor on the PLCA coordinator made run-time retuning of standard burst settings safe. It gave 0 safety deadline misses in 480 governed OMNeT++ runs, at deadlines from 10 to 30 ms, while the same policy without it reached 35.3 ms against 20 ms and conventional PLCA missed 20 % at 90 % load.

The adaptive layer delivered less than planned. Adaptation cut p99 by 16.6 % at one load in one scenario, at a 4.6 % cost in the mean, and gave no p99 gain on bursty traffic. A 596-byte int8 network came close to an oracle at predicting state, yet no better than a moving average once in the loop. Real CAN traces showed the benefit of the governor only at tight deadlines with bulk traffic on the bus, and those tight deadlines needed the non-standard meter.

Future work, in priority order:

1. Build the governor on 10BASE-T1S evaluation boards, measure real timing, and test how burst settings reach followers.
2. Add fault models and policing of nodes that break their budget.

3. Find out why non-critical traffic pays, with an experiment that isolates the cause.
4. Repeat the replay on AutoHack and can-train-and-test for more vehicles.

References

All DOIs below were checked against the Crossref registry on 4 November 2026. Unmarked items were read directly. Items marked (abstract) were read as abstracts only, and items marked (not read) were confirmed as records only; read those in full before submission.

1. IEEE Std 802.3cg-2019, IEEE Standard for Ethernet, Amendment 5: Physical Layer Specifications and Management Parameters for 10 Mb/s Operation and Associated Power Delivery over a Single Balanced Pair of Conductors, doi:10.1109/IEEESTD.2020.8982251. (not read)
2. P. Beruto and A. Orzelli, 802.3cg draft 2.0 PLCA (Clause 148) Overview, IEEE 802.3 Plenary, July 2018.
3. M. Jochim and Y. Kim, A Call for Interest on 10BASE-T1S PLCA Latency Reduction Study, IEEE 802.3 CFI, November 2025.
4. M. Jochim, K. Shazzad and Y. Kim, IEEE 802.3 NEA 10BASE-T1S Differentiated Services, October 2025.
5. D. Pannell, Priority Support for PLCA, IEEE 802.3cg, September 2018.
6. X. Li, J. Villanueva, X. Guo and J. Migge, Overcoming QoS Challenges in a Full Automotive Ethernet Architecture, IEEE Ethernet & IP @ Automotive Technology Day, October 2025.
7. J. Min and Y. Park, Performance Enhancement of In-Vehicle 10BASE-T1S Ethernet Using Node Prioritization and Packet Segmentation, IEEE Access, vol. 10, pp. 103286-103295, 2022, doi:10.1109/ACCESS.2022.3209824. (abstract)
8. J. Min and Y. Park, Interrupt-Enabled Physical Layer Collision Avoidance for 10BASE-T1S Automotive Ethernet, IEEE Trans. Vehicular Technology, vol. 73, no. 5, pp. 6083-6092, 2024, doi:10.1109/TVT.2023.3336843. (abstract)
9. F. Cerruto, L. Leonardi, L. Lo Bello and G. Patti, Performance Assessment of Ethernet 10BASE-T1S Networks in Automotive Applications, IEEE VTC2025-Spring, pp. 1-5, 2025, doi:10.1109/VTC2025-Spring65109.2025.11174892. (abstract)
10. K. Kim, E. Choi and J.-W. Choi, Network Performance Evaluation of IEEE 802.3cg, J. Korean Institute of Communications and Information Sciences, vol. 45, no. 4, pp. 706-711, 2020, doi:10.7840/kics.2020.45.4.706. (abstract)
11. R. L. Cruz, A calculus for network delay, Part I: Network elements in isolation, IEEE Trans. Information Theory, vol. 37, no. 1, pp. 114-131, 1991, doi:10.1109/18.61109. (not read)

12. J.-Y. Le Boudec and P. Thiran, Network Calculus, LNCS 2050, Springer, 2001, doi:10.1007/3-540-45318-0. (not read)
13. R. I. Davis, A. Burns, R. J. Bril and J. J. Lukkien, Controller Area Network (CAN) schedulability analysis: Refuted, revisited and revised, Real-Time Systems, vol. 35, no. 3, pp. 239-272, 2007, doi:10.1007/s11241-007-9012-7. (abstract)
14. S. S. Saha, S. S. Sandha and M. Srivastava, Machine Learning for Microcontroller-Class Hardware: A Review, IEEE Sensors Journal, vol. 22, no. 22, pp. 21362-21390, 2022, doi:10.1109/JSEN.2022.3210773. (abstract)
15. L. Capogrosso, F. Cunico, D. S. Cheng, F. Fummi and M. Cristani, A Machine Learning-oriented Survey on Tiny Machine Learning, arXiv:2309.11932, 2023. (abstract)
16. M. E. Verma et al., A comprehensive guide to CAN IDS data and introduction of the ROAD dataset, PLOS ONE, vol. 19, no. 1, e0296879, 2024, doi:10.1371/journal.pone.0296879.
17. Y. Song, S. Ahn, S. Baek, H. Kim, H. K. Kim and S. Jeon, AutoHack: A Physically Verified Multi-Bus CAN Dataset for Intrusion Detection System Evaluation, USENIX VehicleSec, 2026.
18. B. Lampe and W. Meng, can-train-and-test: A Curated CAN Dataset for Automotive Intrusion Detection, arXiv:2308.04972, 2023.
19. D. A. do Nascimento, S. Bondorf and D. R. de Sousa Campelo, Modeling and Analysis of Time-Aware Shaper on Half-Duplex Ethernet PLCA Multidrop, IEEE Trans. Communications, vol. 71, pp. 2216-2229, 2023, doi:10.1109/TCOMM.2023.3246080. (abstract)
20. J. Ahn and P. Park, Periodic PLCA: Deterministic and Bandwidth Efficient Medium Access for Ethernet Multidrop Networks, IEEE Access, vol. 14, pp. 141054-141068, 2026, doi:10.1109/ACCESS.2026.3732710. (abstract)
21. F. Cerruto, L. Leonardi, L. Lo Bello and G. Patti, Achieving Quality of Service Differentiation in Ethernet 10BASE-T1S nodes, IEEE ETFA, pp. 1-4, 2025, doi:10.1109/ETFA65518.2025.11205774. (abstract)
22. F. Cerruto, L. Leonardi, L. Lo Bello and G. Patti, A Software Component for Real-Time Automotive Communications Over Ethernet 10BASE-T1S, IEEE Trans. Industrial Informatics, vol. 22, pp. 9467-9477, 2026, doi:10.1109/TII.2026.3701238. (abstract)
23. Y. Kim and M. Jochim, PLCA Multidrop Latency, Jitter, and Scalability Enhancements, IEEE 802.3 Call for Interest, July 2026.
24. W. Hellemans, J. Hamborg, T. Lauser, M. M. Rabbani, B. Preneel, C. Krauß and N. Mentens, CarDS: Controller Area Network and Automotive Ethernet Realistic Data Set, IEEE ACSAC, pp. 798-814, 2025, doi:10.1109/ACSAC67867.2025.00069. Licensed CC BY-SA 4.0.
25. IC Plus Corp., European patent application EP 4 401 363 A1, filed March 2023.

26. onsemi, NCV7410 and NCN26010 10BASE-T1S controller product information, Precedence Mode. https://www.onsemi.com/products/interfaces/ethernet-controllers/ncv7410
27. IEEE P802.3da Task Force, 10 Mb/s Single Pair Multidrop Segments Enhancement. https://ieee802.org/3/da/ (not read)
28. Ciraolo, Automotive Ethernet: Construction and Performance Analysis of a 10BASE-T1S Network, MSc thesis, Politecnico di Torino, 2024. https://webthesis.biblio.polito.it/30927 (record only)